In [2]:
```
import numpy as np
import string
import tensorflow as tf
import matplotlib.pyplot as plt

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.utils import to_categorical
from numpy.linalg import norm
```

In [4]:
```
# Sample
text = """
The quick brown fox jumps over the lazy dog.
The fox is quick and the dog is lazy.
Machine learning is a part of artificial intelligence.
Deep learning is a part of machine learning.
Artificial intelligence is changing the world.
"""

# Convert text to lowercase and tokenize
words = text.lower().split()

print("Total words:", len(words))
print("Words:")
print(words)
```

```
Total words: 40
Words:
['the', 'quick', 'brown', 'fox', 'jumps', 'over', 'the', 'lazy', 'dog.', 'the',
'fox', 'is', 'quick', 'and', 'the', 'dog', 'is', 'lazy.', 'machine', 'learning',
'is', 'a', 'part', 'of', 'artificial', 'intelligence.', 'deep', 'learning', 'is',
'a', 'part', 'of', 'machine', 'learning.', 'artificial', 'intelligence', 'is', 'c
hanging', 'the', 'world.']
```

In [5]:
```
import string

# Remove punctuation
words = [
    word.strip(string.punctuation)
    for word in words
]

# Remove empty words
words = [word for word in words if word]

print("Cleaned words:")
print(words)
```

```
Cleaned words:
['the', 'quick', 'brown', 'fox', 'jumps', 'over', 'the', 'lazy', 'dog', 'the', 'f
ox', 'is', 'quick', 'and', 'the', 'dog', 'is', 'lazy', 'machine', 'learning', 'i
s', 'a', 'part', 'of', 'artificial', 'intelligence', 'deep', 'learning', 'is',
'a', 'part', 'of', 'machine', 'learning', 'artificial', 'intelligence', 'is', 'ch
anging', 'the', 'world']
```

In [6]:
```
# Create vocabulary
vocab = sorted(set(words))

vocab_size = len(vocab)
```

```
# Word to index
word_to_index = {word: i for i, word in enumerate(vocab)}

# Index to word
index_to_word = {i: word for i, word in enumerate(vocab)}

print("Vocabulary Size:", vocab_size)
print("\nVocabulary:")
print(vocab)
```

```
Vocabulary Size: 20

Vocabulary:
['a', 'and', 'artificial', 'brown', 'changing', 'deep', 'dog', 'fox', 'intelligen
ce', 'is', 'jumps', 'lazy', 'learning', 'machine', 'of', 'over', 'part', 'quick',
'the', 'world']
```

In [7]:

```
# Context window size
window_size = 2

X = []
y = []

# Generate training examples
for i in range(window_size, len(words) - window_size):

    context_words = (
        words[i-window_size:i] +
        words[i+1:i+window_size+1]
    )

    target_word = words[i]

    # Convert context words to one-hot vectors
    context_vector = np.zeros(vocab_size)

    for word in context_words:
        context_vector[word_to_index[word]] += 1

    # Average the context vectors
    context_vector = context_vector / len(context_words)

    # Target word index
    target_index = word_to_index[target_word]

    X.append(context_vector)
    y.append(target_index)

X = np.array(X)
y = np.array(y)

# One-hot encode target
y = to_categorical(y, num_classes=vocab_size)

print("Input shape :", X.shape)
print("Target shape:", y.shape)
```

```
Input shape : (36, 20)
Target shape: (36, 20)
```

In [8]:
```
print("Number of training examples:", len(X))

for i in range(min(5, len(X))):
    print("\nExample", i + 1)
    print("Input vector:", X[i])
    print("Target word :", index_to_word[np.argmax(y[i])])
```

```
Number of training examples: 36

Example 1
Input vector: [0.   0.   0.   0.   0.   0.   0.   0.25 0.   0.   0.25 0.   0.
0.
 0.   0.   0.   0.25 0.25 0.  ]
Target word : brown

Example 2
Input vector: [0.   0.   0.   0.25 0.   0.   0.   0.   0.   0.   0.25 0.   0.
0.
 0.   0.25 0.   0.25 0.   0.  ]
Target word : fox

Example 3
Input vector: [0.   0.   0.   0.25 0.   0.   0.   0.25 0.   0.   0.   0.   0.
0.
 0.   0.25 0.   0.   0.25 0.  ]
Target word : jumps

Example 4
Input vector: [0.   0.   0.   0.   0.   0.   0.   0.25 0.   0.   0.25 0.25 0.
0.
 0.   0.   0.   0.   0.25 0.  ]
Target word : over

Example 5
Input vector: [0.   0.   0.   0.   0.   0.   0.25 0.   0.   0.   0.25 0.25 0.
0.
 0.   0.25 0.   0.   0.   0.  ]
Target word : the
```

In [9]:
```
model = Sequential([
    Dense(50, input_shape=(vocab_size,), activation='relu'),
    Dense(vocab_size, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_2 (Dense)	(None, 50)	1,050
dense_3 (Dense)	(None, 20)	1,020

Total params: 2,070 (8.09 KB)

Trainable params: 2,070 (8.09 KB)

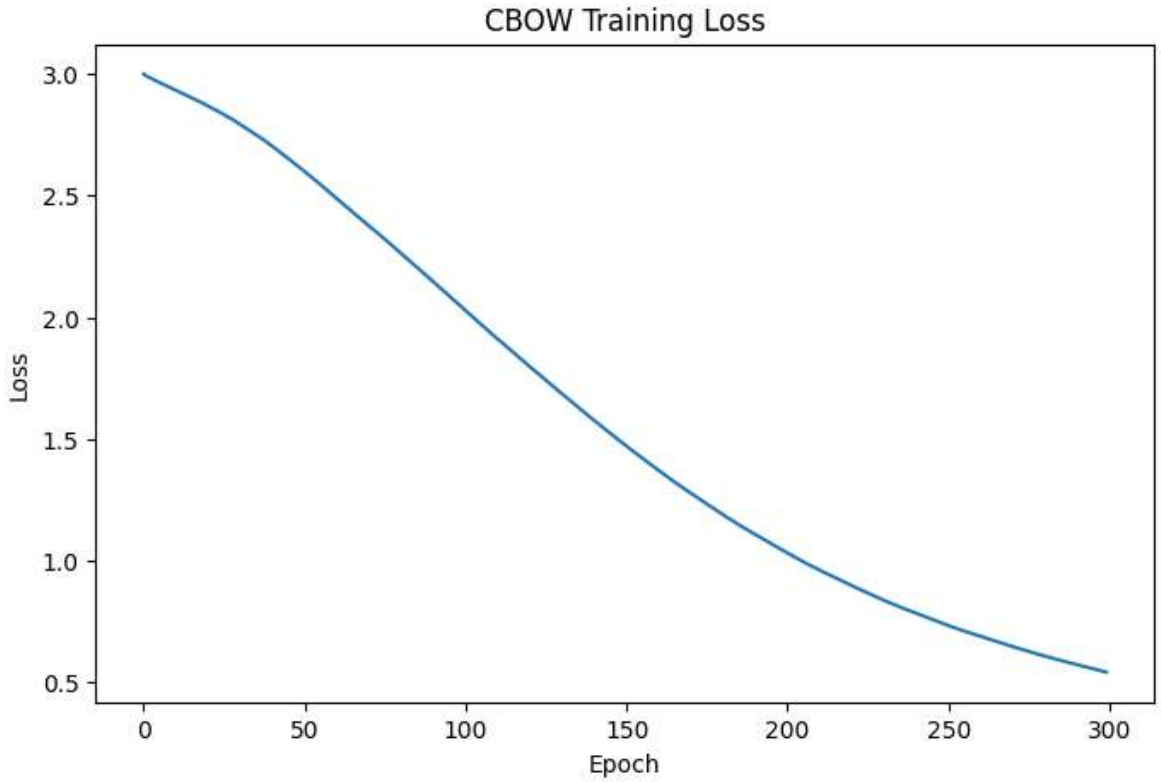
Non-trainable params: 0 (0.00 B)

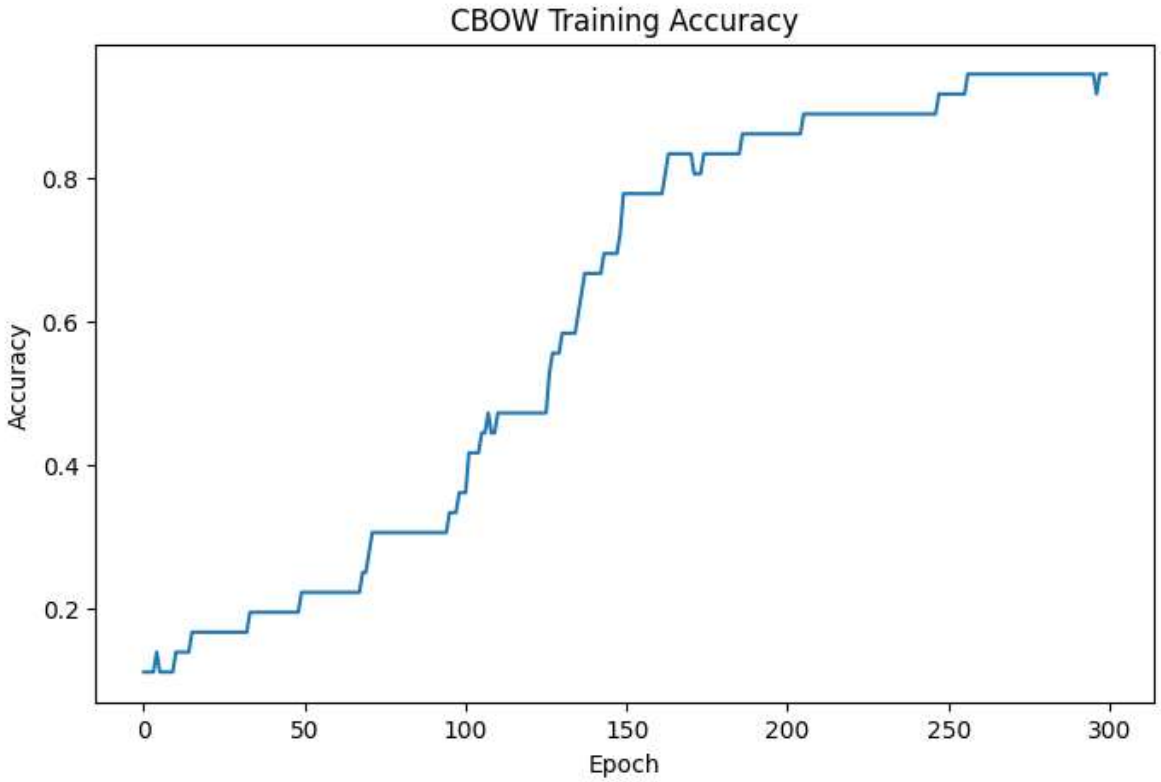
In [10]:
```
history = model.fit(
    X,y, epochs=300, batch_size=16,verbose=1
)
```

In [11]:
```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))
plt.plot(history.history['loss'])
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("CBOW Training Loss")
plt.show()

plt.figure(figsize=(8, 5))
plt.plot(history.history['accuracy'])
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("CBOW Training Accuracy")
plt.show()
```





In [12]:
```python
def predict_word(context_words):

    context_vector = np.zeros(vocab_size)

    for word in context_words:
        if word in word_to_index:
            context_vector[word_to_index[word]] += 1

    context_vector = context_vector / len(context_words)

    prediction = model.predict(
        np.array([context_vector]),
        verbose=0
    )

    predicted_index = np.argmax(prediction)

    return index_to_word[predicted_index]
```

In [14]:
```python
context = ["fox","and","the","is"]

predicted_word = predict_word(context)

print("Context words:", context)
print("Predicted word:", predicted_word)
```

```
Context words: ['fox', 'and', 'the', 'is']
Predicted word: quick
```

In [ ]: